2026 Hello World Dev Conf

# AI 導入之後

## 傳統開發團隊的角色鬆動與開發新節奏



**陳仁傑**
國泰金控 資深工程師

2026.10.14 | Room A

先講結論

# AI 寫 code 很快，
# 但我們團隊變最多的不是速度

導入 AI 大半年，真正變了的是這兩件事

## 角色

每個人能做的事變多了，
角色的界線開始鬆動

## 節奏

從討論需求到上線，
做事的順序都不一樣了

# 大綱

1 這個專案和團隊

2 遇到的困難，和我們怎麼解決

3 AI 帶來的新節奏

4 做完之後的心得

# 團隊 6 個人，一半沒有程式開發背景

**工程師** 組長
10 年經驗

**工程師**
1 年經驗

**工程師**
1 年經驗

**業務分析** BA
沒有程式開發背景

**專案管理** PM
沒有程式開發背景

**介面設計** UX
沒有程式開發背景

再加 1 位

## AI 隊友

一個 AI Agent，
幫團隊處理重複的雜事

實驗探索中

● 3 位有開發底子　● 3 位沒有程式開發背景

**金融業的專案，要一路做到上線，不是做個 PoC 就結束**

# 一人負責一個模組，每個人都要寫程式

**一個服務**　對使用者來說，就是一套系統

**用微服務的設計，拆成 5 個模組**

| 模組 A | 模組 B | 模組 C | 模組 D | 模組 E |
| --- | --- | --- | --- | --- |
| 一人負責 | 一人負責 | 一人負責 | 一人負責 | 一人負責 |

**也因為這樣，沒有程式開發背景的同仁也得自己寫程式，後面的困難就是從這裡開始的**

# 我們用了哪些工具

開發

## Claude Code

每個人都用它開發，
從需求草稿到寫程式、測試

協作

## GitHub

開票、排 Sprint、發 PR、review
都在上面，用的是付費的 Team 方案

溝通

## Discord

平常溝通、討論都在這裡

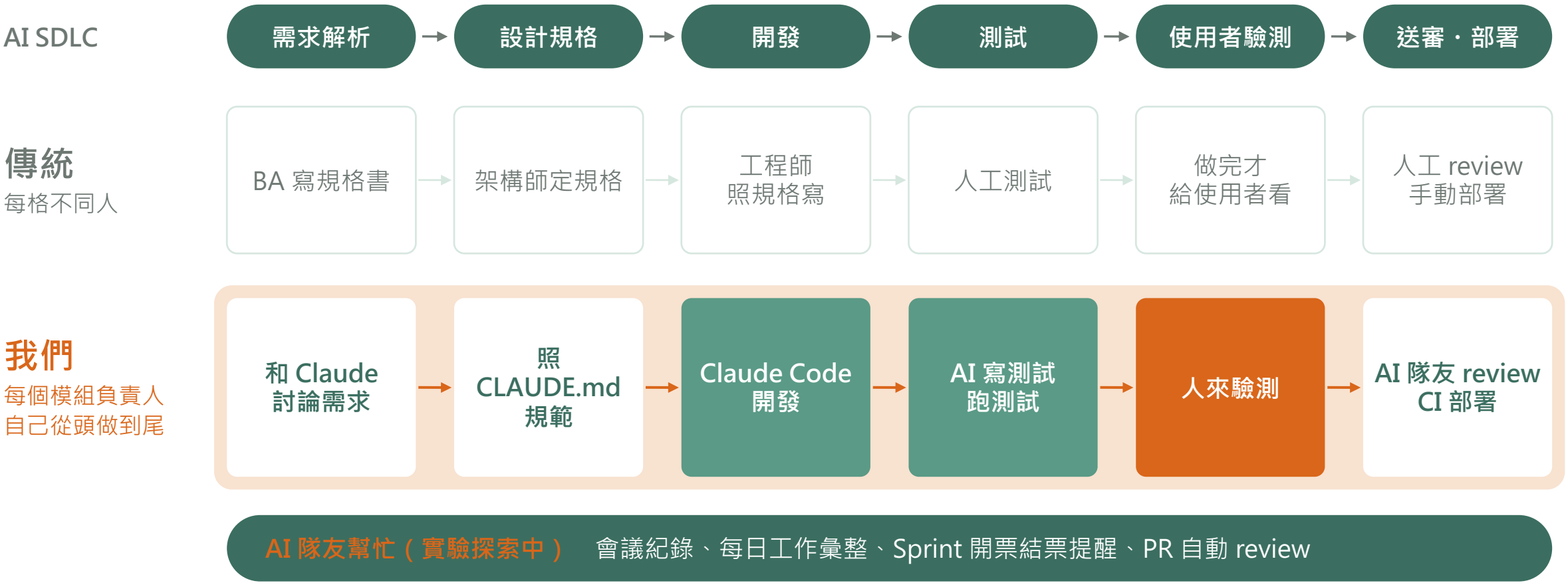
AI 隊友

## AI Agent

實驗探索中

負責團隊重複的雜事；
只做整理和建議，決定都由人來做

# 全員開發，每個人都要自己走完整條 SDLC



# 兩群人，卡的地方不一樣

## 沒有程式開發背景的視角

業務分析、專案管理、介面設計

**困難一**

不知道自己在幹嘛，AI 做完了，
卻說不出它是怎麼做的

**困難二**

要學的東西很多，Git、框架、shell、
GitHub 流程都得從頭學

**解法是一小步一小步做，**
**把 AI 當老師**

## 有程式開發背景的視角

工程師

**困難**

工具都會，但每個人寫法不一樣，
JDBC、API、框架、架構各寫各的

**解法是先給規範，做完再檢查，**
**慢慢長出 Harness**

沒有程式開發背景的視角

# 「我不知道自己在幹嘛」

**交代給 AI**
請 AI 做一件事

→

**中間發生什麼事？**
AI 怎麼決定、為什麼這樣做，完全沒有 sense

→

**AI 交回來**
AI 做完了

## 就像用導航

跟著導航開到目的地，
但每個路口為什麼這樣轉，其實不知道

## 之後很難維護

出問題看不懂也修不了，
自己也沒學到東西，這就是 vibe coding

沒有程式開發背景的視角

# 一小步一小步做，把 AI 當老師

**一大步**
不建議

一次丟一大件事給 AI，要看懂的東西太多，根本跟不上

**小步**
我們的做法

第 1 步 → 第 2 步 → 第 3 步 → 第 4 步

懂了再往下　懂了再往下　懂了再往下　懂了再往下

## 請 AI 當老師

每一步都請它講
在做什麼、為什麼這樣寫

## 問有開發背景的同事

AI 說不清楚的地方，找人一起看

## 理解到夠用就好

懂到能把手上的事做完就好

沒有程式開發背景的視角

# 要學的東西太多，以 UX 同仁為例

分到的是比較單純的前端模組，但下面這些都得邊做邊學

## Git

Git 是什麼、分支怎麼開，
AI 會用到，只好從頭學

## Shell 指令與目錄

下指令、切目錄、看設定檔

**UX 的本職**
介面設計

## 軟體框架與架構

專案用的框架、分層、模組怎麼切

## GitHub 的運作流程

GitHub Project、看板、
issue、PR 怎麼走

**這些都不在原本的專業裡，要補的真的很多**

沒有程式開發背景的視角

# 該學的還是要學，不然就是在迷航

沒花時間學

## 在迷航（vibe coding）

- AI 做完了，不知道怎麼做到的
- 出問題看不懂、修不了
- 完全不知道自己在幹嘛

有花時間學

## 知道自己在做什麼

- 知道每一步在做什麼
- 看得出 AI 有沒有走偏
- 是你在用 AI，不是被 AI 帶著走

**用 AI 的順序**

**① 心裡先有概念**
知道接下來要 AI 做什麼

→

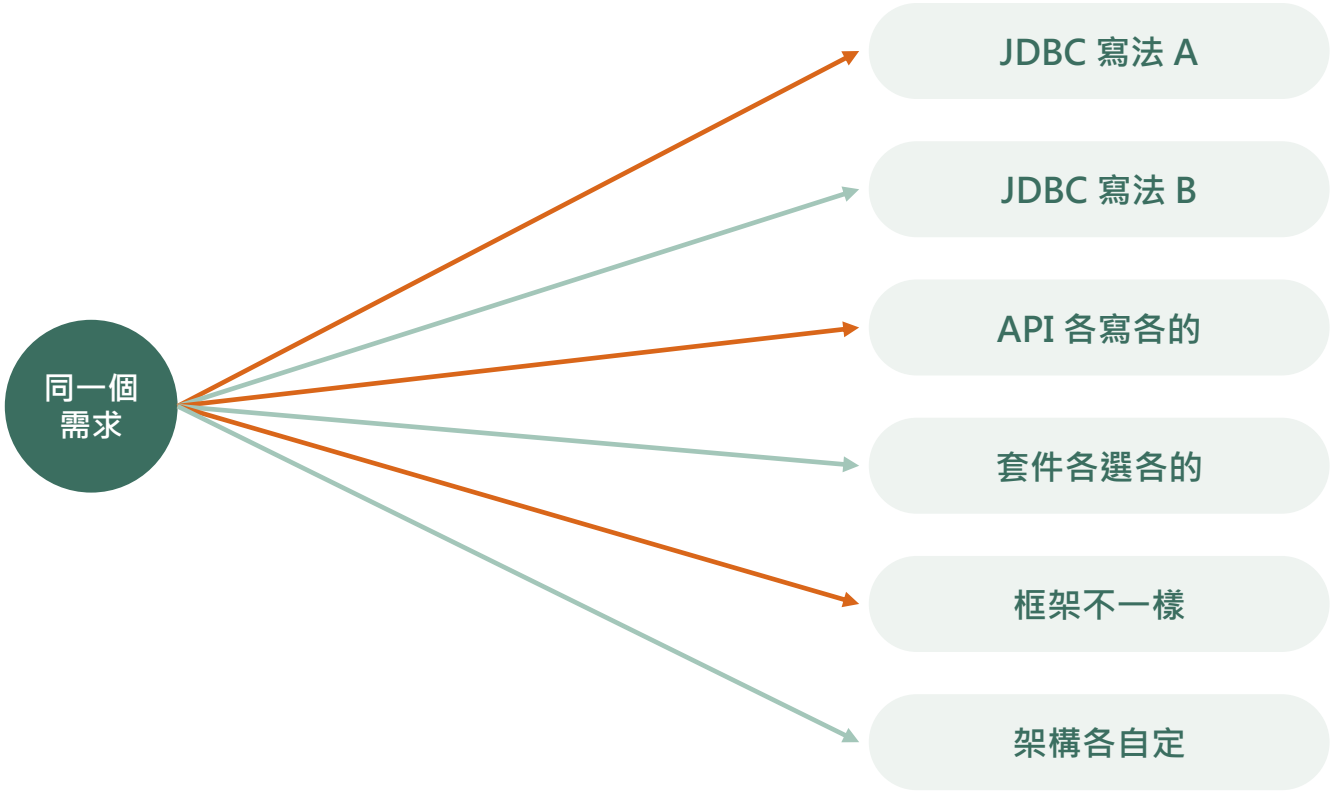
**② 讓 AI 做**
照你的想法做出來

→

**③ 回頭驗證**
看結果跟你想的一不一樣

**AI 是幫你把想法做出來、再驗證；想法要先在你腦袋裡，所以該學的省不掉**

# 工具都會，但每個人的寫法都不一樣




## 工具和軟體知識都懂

Git、框架、指令都沒問題，
卡的不是會不會

## 沒有規範，就各寫各的

一開始規範還沒建好，
每個人的 AI 照自己的方式寫，
5 個模組可能 5 種寫法

# 前饋，先把規範放進 metadata workspace

| | |
|---|---|
| metadata-workspace/ | 大家都從這裡開 Claude Code |
| CLAUDE.md | 團隊共同的規範和架構共識，大家的 AI 開工前都會讀 |
| CLAUDE.local.md | 個人習慣寫這裡，只留在自己電腦 |
| .claude/ | 團隊共用的 AI 設定（進版控） |
| rules/ | 架構規則，碰到相關檔案才會載入 |
| skills/ | 共用 skill，有人寫好，大家 pull 下來就能用 |
| 模組-A/ | 各自負責的模組，一人一個 |
| CLAUDE.md | 模組自己的補充規則，但不能跟團隊規範衝突 |
| 模組-B/ … 模組-E/ | 其他四個模組，結構一樣 |

### 同一份 context

大家都從最外層開 Claude Code，
AI 讀到的規範一樣，寫出來就不會差太多

### 團隊的和個人的分開

團隊規則進版控，
個人習慣留在自己電腦，不會互相影響

### 工具不用自己裝

共用 skill 放在外層，pull 就有；
要加新工具就發 PR

# CLAUDE.md 裡寫的，是團隊的架構共識

| 寫進 CLAUDE.md 的架構方法（前饋） | | AI 寫完後怎麼確認有照做（後饋） |
|---|---|---|
| **Clean Architecture**<br>四層分層，核心邏輯不依賴資料庫與框架 | → | 硬護欄 ArchUnit 檢查分層，Spring Modulith 檢查模組邊界，違反就會失敗 |
| **CQRS**<br>讀和寫分開處理 | → | 硬護欄 AI 隊友自動 review，照團隊規則看讀和寫有沒有分開，再由人確認 |
| **Event Sourcing**<br>只有需要留下狀態歷史的地方才用，不強制 | → | 軟規範 寫在 CLAUDE.md 當參考，由人判斷要不要用 |

# 規則邊做邊補，AI 慢慢有了「開發慣性」

## 初期

**規範還少**

其實就是 vibe coding，
AI 照一般寫法寫

## 過程

**邊做邊補規則**

發現走偏，就補進規範，
或寫成自動檢查

## 後期

**開發慣性**

規則和程式碼夠多之後，AI
會照專案原本的寫法寫

### 慣性也有壞處

好的寫法會被照抄，
早期沒修掉的錯也會被照抄。
發現走偏要趕快修，
拖越久被複製越多

**現有的程式碼，
本身就是 AI 的範本**

有程式開發背景的視角

# 我們的 Harness，每一關都有人或機器把關

AI SDLC

需求解析 → 設計規格 → 開發 → 測試 → 使用者驗測 → 送審・部署

人 50% AI 50% | 人 30% AI 70% | 人 0% AI 100% | 人 10% AI 90% | 人 70% AI 30% | 人 40% AI 60%

**前饋規範**

從 workspace 開 Claude Code 就會先讀

- CLAUDE.md
- skills
- rules

**hook 和 linter**

AI 一動手就自動檢查

**測試和架構檢查**

單元測試

E2E

ArchUnit

Spring Modulith

**人驗測**

看結果對不對

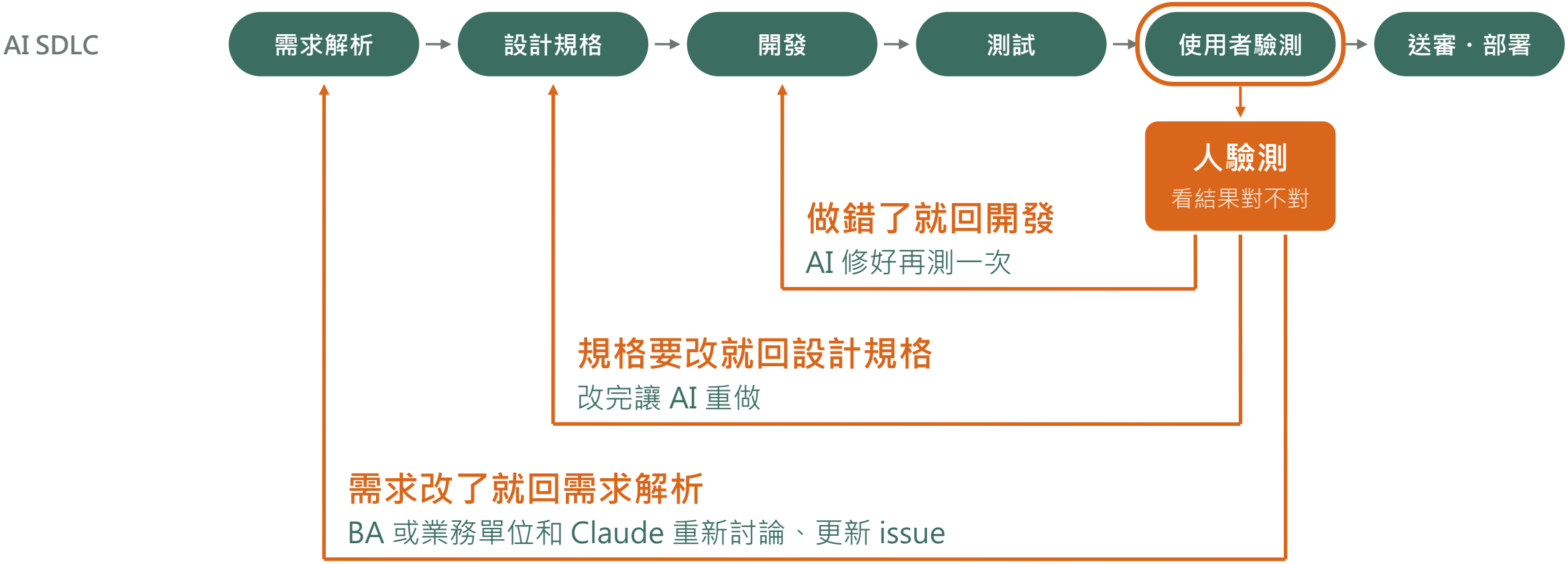
→ 人決定發 PR

→ AI 隊友照 checklist review

→ 人決定合併、部署

踩到新的坑，
就寫回 checklist 和 rules

# 驗測沒過，看是哪種改變就退回哪一格



新節奏

# 需求的探索與驗證，可以「左移」



新節奏

# 螺絲先別鎖死

組 DIY 家具的時候，沒人一開始就把螺絲鎖死，因為後面常會發現拼錯、拼反



## 鬆

### 需求探索（發散）

還不知道要什麼，就讓 AI 多做幾種，看了才知道想要哪個



## 半鎖

### 需求驗證（收斂）

有想法了，用很像真的草稿確認對不對，中途改主意很正常



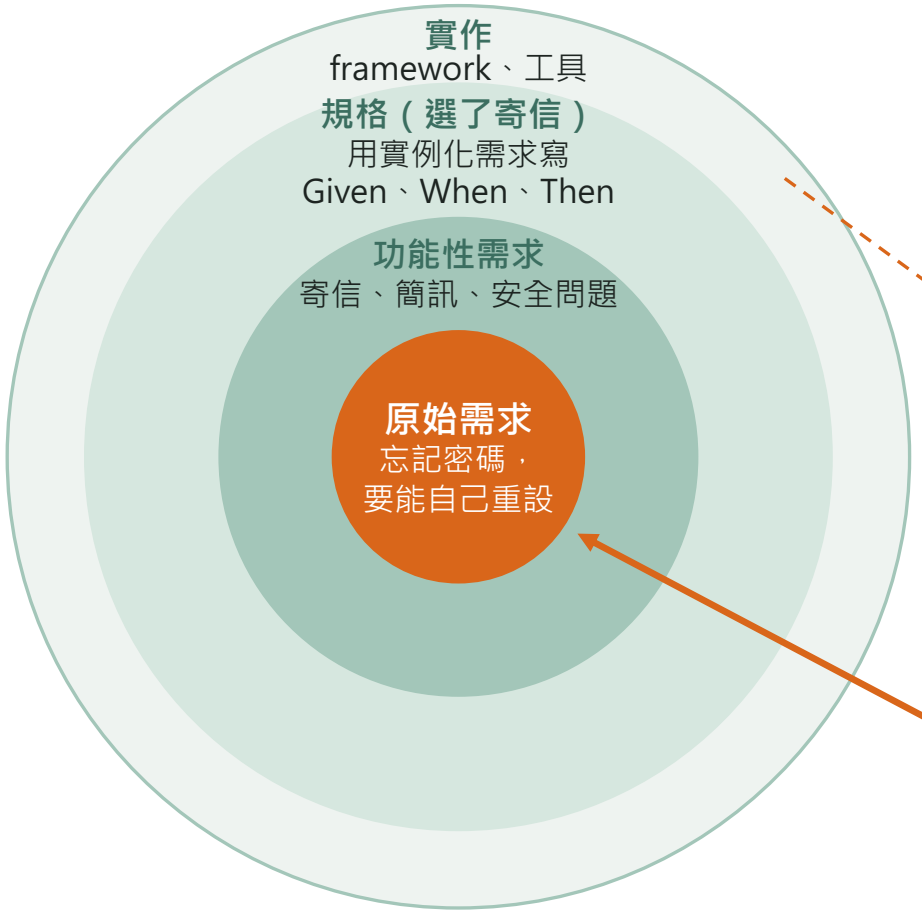
## 鎖緊

### 定案再落地

需求確定了，再交給 SDD、Harness 把實作做扎實

**需求會變，是因為你自己的想法會變。** 草稿不用寫到 production code 的等級

# 需求和規格脫鉤，越裡面越不會變




## 和 Clean Architecture 是同一個道理

**Clean Architecture**
最裡面是 Entities，用哪個 framework 實作都不影響它

**實例化需求，把需求轉成規格的橋樑**
不寫長篇規格書，用具體的例子當規格

| | |
|---|---|
| Given | 使用者忘記密碼 |
| When | 輸入 Email 並送出 |
| Then | 系統發送 6 碼 OTP，15 分鐘過期 |

**確認需求的工具**
artifact、vibe coding、Miro、UML，不用等規格和實作

新節奏

# 文件只寫 code 看不出來的，開發才跑得快

改一次規格，要動幾個地方

以前 — 程式碼 | 規格書 | API 文件 | 設計書

常常沒跟上，AI 還會照舊的寫

現在 — 程式碼

API 文件由工具從 code 產生；需求和取捨的紀錄，規格換了也不用改

## 要寫 不常變的

- 需求，還有設計的取捨（為什麼這樣做）
- 一份專案規則，給 AI 當入口
- 從 code 自動產生的文件
- 對外的介面契約、稽核法遵要的文件，照規定寫

## 不寫 常變，而且 code 已經寫了的

- 每支 API、每個 class 的說明
- 跟 code 重複的文件
- 寫得很細的設計書

實驗探索階段

# 第七位隊友，讓 PM 不用再追進度



## AI 隊友

AI Agent，天天上班



6 個人加 1 位 AI 隊友

### 會議紀錄

站會錄音自動轉成文字紀錄

### 每日彙整

出缺勤與每日工作彙整

### 流程提醒

Sprint 開始提醒開 issue；
結束提醒關 issue，
或移到下個 Sprint

### PR review

每個 PR 自動 review，
照團隊規則一條一條對，
只給建議，合併由人決定

實驗探索階段

# AI 隊友給建議，人覆核、人決定、人負責

## AI 給建議

- 範圍事先設計好，只做團隊內部重複的雜事
- 該做什麼，寫成有版控的規則和流程

## 人覆核

- AI review 只給建議、附證據，不擋 merge
- 使用者驗測，一定由人來看

## 人決定

- 發 PR、合併、部署，都由人決定
- 被檢查擋下時，AI 不自己繞過，交回給人

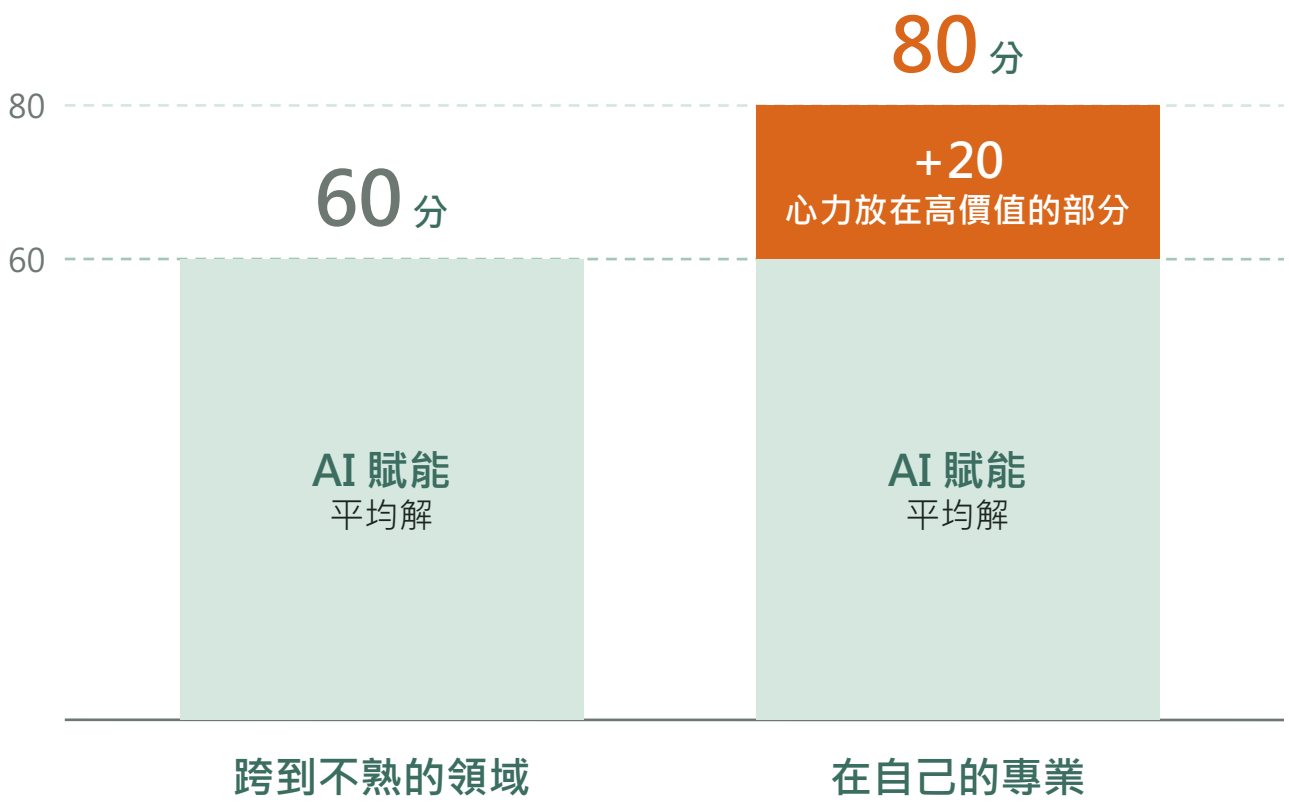
## 人負責

- 每個模組都有負責人，AI 的產出由他把關
- 決定都留在 issue 和 PR 上，事後查得到

**我們的原則** AI 可以幫很多忙，但最後的判斷和責任，一定在人身上

心得

# 在自己的專業上，AI 是加速器，也是放大器




**AI 賦能，誰來用都有 60 分**

不熟的領域也做得出及格的東西，但就是一般水準

**低價值的部分交給 AI，整個過程變快**

重複的、照規則就能做的事讓 AI 執行，這是加速

**省下的心力，花在高價值的部分**

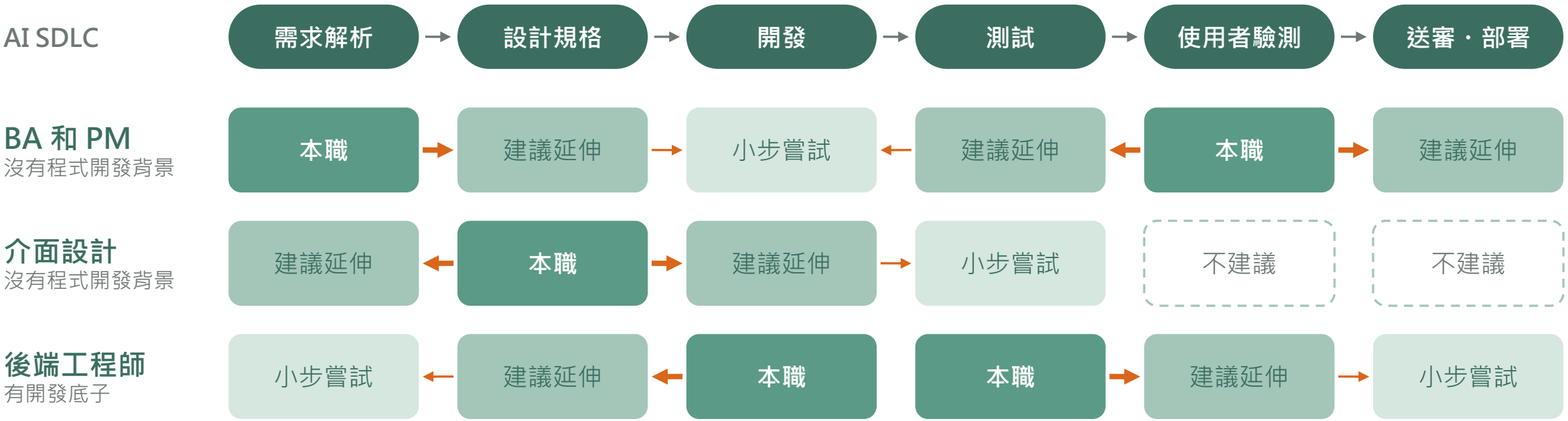
判斷、取捨、設計自己來，這才是被放大的地方

**差的那 20 分，就是前備知識**

懂這個領域，才分得出哪些是低價值、哪些是高價值

心得

# 角色的邊界鬆動了，往相近的領域延伸效果最好




## 「相近」看的是業務知識，不是職稱

照 SDLC 來看，BA 寫程式算跨很遠，但程式裡的業務邏輯 BA 最熟，所以靠 AI 也能主導開發

心得

# 今天帶走的四件事

1

## 從 vibe coding 長出 Harness

踩過的坑寫成規則，大家的 AI 寫法就會一致

二、困難與解法 p13–18

2

## 需求探索與驗證左移

用 AI 草稿快速弄清楚自己要什麼，螺絲先別鎖死

三、新節奏 p19–20

3

## 需求與規格脫鉤、文件輕量化

需求留著，規格可以換；文件只寫程式碼看不出來的

三、新節奏 p21–22

4

## 自己的專業被 AI 加速和放大，跨到不熟的領域靠它賦能

前備知識決定 AI 能幫你做到 80 還是 60；往相近的領域延伸最划算

四、心得 p25–26

心得

# 回頭看，今天講的每件事其實是同一件事

## AI 隊友的角色與權限

**角色**　負責什麼、不負責什麼

**工具與權限**　它能碰什麼

**像第七位隊友**　角色與權限都是設計出來的

## AI 看得到的東西

**前備知識**　60 分和 80 分的差距　80 / 60

**專案規則**　只寫 code 讀不出來的　文件輕量化

**既有 codebase**　AI 的開發慣性　開發慣性

**大家讀同一份**　每個人的 AI 看到同樣的東西　workspace

心得

# 「什麼都無法捨棄的人，就絕對沒辦法改變任何事」




## 放下了

**自己寫每一行 code** p17
開發交給 AI，人守住需求和驗測

**需求一次定死** p20
螺絲先別鎖死，看了想法會變很正常

**厚厚的規格書** p22
文件只寫 code 看不出來的

**只做自己的本職** p26
往相近的領域延伸

最後一句話

# 以前，我們設計軟體架構。
# 現在，我們設計 AI agent，
# 以及它看得到的 context。



## 謝謝聆聽

陳仁傑 | 國泰金控 資深工程師

國泰金控 | BETTER TOGETHER

提問時間

# Q&A

陳仁傑 | 國泰金控 資深工程師

國泰金控 | BETTER TOGETHER